SIMATS ENGINEERING

Department of Information Technology

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Field	Details	Field	Details
Student Name	M.Greeshma	Register Number	192524195
Department / Section	Information Technology	Academic Year / Semester	2026-2027
Faculty Name	Dr.R.Latha	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	3/11/2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Characterise the Virtualization Architectures	5		
2	Identify the Dominant Overhead Source	2		

Task	Title	Max	Awarded	Evidence / evaluation focus
3	Build a Quantitative Performance Model	6		
4	Analyse Performance Under Different Conditions	4		
5	Security Threat and Vulnerability Analysis	3		
6	Formulate Isolation–Performance Trade-off	3		
7	Evaluate Mitigation Strategies	4		
8	Implement and Test	4		
9	Record Experimental Performance	3		
10	Compare Theory with Experiment	4		
11	Recommend and Defend Final Architecture	4		
	Day-4 Design Challenge	8		

	TOTAL			
		50		

OVERALL REMARKS

Overall, the assignment reflects good analytical ability, technical understanding and problem-solving skills in cloud computing and virtualization. The proposed architecture balances performance, security, resource efficiency and future scalability. The work can be further strengthened through verified experimental results, consistent benchmark comparisons and practical validation of the recommended security controls.

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; X = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three virtualization architectures, layered diagrams and trust boundaries		
2	Dominant overhead source identified and explained for each approach		
3	CPU, memory, I/O, network and density models with assumptions		
4	Best, typical and worst operating-condition analysis		

Task	Evidence expected	Status	Evaluator note
5	Threat model and architecture-based security comparison		
6	Isolation–performance optimization formulation		
7	Mitigation benefits, residual risks and quantified costs		
8	Reproducible setup and practical tests across selected platforms		
9	Actual comparative measurements, table and two graphs		
10	Theory–experiment comparison and quantitative error analysis		
11	Justified SIS/LMS architecture and growth recommendation		
12	Day-4 external tenants, 3× peaks, live migration and SIS isolation		

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Architecture diagrams		
Performance model		

Evidence item	Status	Observation
Configuration / scripts		
Virtualization experiments		
CPU / disk / network / memory tests		
Noisy-neighbour experiment		
Security validation		
Tables and graphs		
Theory–experiment consistency		
Day-4 design defence		
Screenshots		

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 12 tasks answered	☐Yes ☐Partly ☐No
Three virtualization approaches compared	☐Yes ☐Partly ☐No
Layered diagrams and trust boundaries included	☐Yes ☐Partly ☐No
Quantitative virtualization models included	☐Yes ☐Partly ☐No
Threat analysis and mitigations included	☐Yes ☐Partly ☐No

CLOUD COMPUTING AND BIG DATA

ASSIGNMENT 2

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Assignment Title: Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud (Total Marks: 50)

Duration: 1 week

Scenario: EduCloud Virtualization Platform

A university runs a private cloud, **EduCloud**, in its data centre. EduCloud currently consists of:

- **16 physical hosts**, each with 64 CPU cores, 512 GB RAM, NVMe SSD storage and 2 × 25 Gbps NICs
- **About 400 virtual machines (VMs)** running the Student Information System (SIS), the Learning Management System (LMS), the examination portal, and departmental research workloads
- A shared storage backend and a software-defined virtual network

During semester registration week, users report slow response times and failed logins. A recent internal security audit also found that VMs from different departments share the same physical hosts, that some VMs run outdated guest kernels, and that VM images are copied freely between teams.

You are the cloud architect. You must **investigate, model, measure and compare** virtualization architectures for EduCloud, identify their performance bottlenecks and security weaknesses, and recommend a design that remains practical as the platform grows.

Design Challenge:

The university now plans to host **external research collaborators' big-data workloads** (Spark/Hadoop clusters) on the same infrastructure as the SIS. Re-evaluate your design under these new multi-tenant conditions.

TASK 1: Characterise the Virtualization Architectures (5 Marks)

Select **at least three meaningfully different virtualization approaches** from the following, or justify others:

- Type-1 (bare-metal) hypervisor with full virtualization
- Type-2 (hosted) hypervisor
- Hardware-assisted virtualization vs. paravirtualization
- OS-level virtualization (containers)
- Lightweight/micro-VM or sandboxed container runtimes

Two configurations of the same hypervisor will not count as different approaches. Your selection must include at least one VM-based and at least one container-based approach.

For each approach, provide:

- A layered architecture diagram showing the hardware, host OS or hypervisor, guest OS, runtime and application
- The **trust boundaries** and the component that enforces isolation
- How CPU, memory, storage and network are virtualized
- A brief justification for why this approach is relevant to EduCloud

Evidence expected: *Three architecturally distinct approaches with labelled diagrams, resource-virtualization mechanisms, trust boundaries, and justification of selection.*

TASK 2: Identify the Dominant Overhead Source (2 Marks)

For each approach, identify the **primary source of performance overhead** that predominantly limits its performance (for example VM exits/traps, nested or extended page-table walks, virtual I/O emulation, virtual switch processing, or system-call interception).

Discuss:

- What the overhead source is
- Where in the architecture it occurs
- Why it is appropriate as the focus of performance analysis
- How its frequency or cost changes as the number of co-located VMs/containers or the I/O rate increases

Do not simply name a mechanism. Relate it to the behaviour of the architecture.

Evidence expected: *A justified dominant-overhead identification for each approach, linked to its resource-virtualization path.*

TASK 3: Build a Quantitative Performance Model (6 Marks)

Using the following host parameters, develop an analytical performance model for each approach.

Parameter	Value
Physical cores per host	64
vCPU:pCPU overcommit ratio	1:1, 2:1, 4:1, 8:1
RAM per host	512 GB
Average VM memory demand	8 GB
Cost of one VM exit (assume)	1.5 µs
Native system-call cost (assume)	0.2 µs
Network line rate	25 Gbps

State and justify any additional assumptions. Derive expressions for:

- **CPU:** effective per-VM CPU capacity as a function of overcommit ratio and virtualization overhead (including steal time)
- **Memory:** the maximum number of VMs per host before ballooning, swapping or page-sharing becomes necessary
- **I/O and network:** effective throughput as a function of the number of VMs sharing the physical device
- **Density:** the number of VMs/containers a host can support while meeting a stated service-level target (for example, ≤ 10% performance degradation)

Show the mathematical reasoning. Merely quoting overhead percentages from the literature will not receive full credit.

Evidence expected: *A clear mathematical derivation for each approach, with stated assumptions, parameter values and worked numerical examples.*

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks)

Analyse how each approach behaves under three conditions:

- **Best case:** what workload and placement conditions minimise overhead?
- **Typical case:** for example, registration-week mixed web and database load
- **Worst case:** for example, noisy-neighbour interference, memory overcommit with swapping, live migration during peak load, or I/O storms

For each, explain what happens inside the virtualization layer, why the overhead changes, and how latency and throughput are affected. A table of numbers alone will not be accepted.

Evidence expected: *A justified condition-based analysis supported by your Task 3 model.*

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Construct a threat model for each approach. Discuss:

- The attack surface (hypervisor, device emulation, shared kernel, management plane, VM images and snapshots, virtual network)
- At least **four distinct threat categories**, such as VM escape, cross-VM side-channel attacks, hypervisor or container-runtime vulnerabilities, VM sprawl and unpatched guests, image tampering, insecure live migration, and virtual-network sniffing or spoofing
- For each threat, which approach is more or less exposed and **why**, referring to its architecture and trust boundaries
- The likely impact on the confidentiality, integrity and availability of student records

Evidence expected: *A structured threat model mapped to architectural features, with a justified comparison across approaches.*

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

Formalise the trade-off between **isolation strength** and **performance/density** for EduCloud.

Clearly identify:

- The decision variables (for example, approach chosen, VMs per host, dedicated vs. shared hosts, SMT on/off)

- The objective(s), such as maximising density or minimising cost
- The constraints (for example, performance degradation limit, isolation requirement for SIS, available hosts)
- How these components combine into a cost or decision function

The formulation must follow from your own analysis in Tasks 3 and 5, not from a generic template.

Evidence expected: *A clearly defined trade-off model with identified variables, objective and constraints.*

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Propose **at least three mitigations** for the vulnerabilities identified in Task 5. Examples include disabling SMT, kernel page-table isolation, dedicated hosts for sensitive tenants, secure or sandboxed container runtimes, hardware memory encryption, image signing, and micro-segmentation.

For each mitigation, discuss:

- Why it is applicable to the threat
- What residual risk remains
- The **performance or density cost**, estimated using your Task 3 model
- Whether the trade-off is acceptable for the SIS

Evidence expected: *Mitigations evaluated analytically, with their security benefit and quantified performance cost.*

TASK 8: Implement and Test (4 Marks)

Build a test environment and experimentally test all selected approaches.

- **Platforms:** use any open-source stack, for example KVM/QEMU, Xen or VirtualBox, Docker/Podman, and Kata Containers or gVisor. Nested virtualization on a laptop is acceptable if documented.
- **Scale:** run each approach with approximately $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$ co-located instances (reduce proportionally if hardware-limited, and state the limitation).
- **Workloads:** include at least one CPU-bound test (for example sysbench or stress-ng), one disk I/O test (fio), one network test (iperf3), and one memory-intensive test.
- **Interference test:** run at least one noisy-neighbour experiment.

- **Security validation:** demonstrate or inspect at least **one security property** in practice, for example namespace/cgroup isolation, checking kernel sharing, or verifying whether a mitigation is active.

Ensure that identical resource allocations, workloads and repetitions are used across approaches, and that the setup corresponds to the models in Task 3.

Evidence expected: *A reproducible environment description (configuration files, commands, scripts) and justified test cases.*

TASK 9: Record Experimental Performance (3 Marks)

Collect and present data such as:

- Throughput, latency (mean and 95th percentile), CPU steal time, context switches and VM exits where observable
- Instance start-up time and memory footprint per instance
- Performance degradation versus a bare-metal baseline
- Results with and without at least one security mitigation

Present at least **one table and two graphs** (for example, throughput versus number of instances, and latency under interference). Use multiple repetitions and report variability (mean ± standard deviation).

Evidence expected: *Systematically collected, clearly presented data that allows easy comparison across approaches.*

TASK 10: Compare Theory with Experiment (4 Marks)

Compare your Task 3 and Task 4 predictions with Task 9 measurements.

Discuss:

- Whether the observed degradation trends agree with your model
- Which approach scales better as co-located instances increase
- Quantitative differences between predicted and measured values, and plausible causes (for example caching effects, hypervisor scheduler behaviour, nested virtualization, laptop hardware, or unmodelled I/O paths)
- Whether any measured result changes your security conclusions from Task 5

Interpret the results; do not simply reproduce tables.

Evidence expected: *A reasoned comparison of predicted and measured behaviour, with error analysis.*

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

Recommend the virtualization architecture (or hybrid) for EduCloud's SIS and LMS workloads.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its modelled and experimentally observed performance
- Its security posture and residual risks
- Operational costs such as patching, image management, migration and monitoring
- Limitations and trade-offs
- Whether it remains practical as EduCloud grows from 400 to several thousand instances

Your recommendation must rest on your own analysis and experiments. Choosing "the most secure" or "the fastest" option alone is not sufficient.

Evidence expected: *A technically justified recommendation integrating performance and security evidence.*

DAY-4 DESIGN CHALLENGE

TASK 12: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now host **external research collaborators' Spark/Hadoop workloads** on EduCloud alongside the SIS. The new conditions are:

- Big-data jobs are I/O- and network-intensive and run for hours
- Registration-week peak load is about 3× normal
- Hosts are sometimes patched using live migration of running VMs
- A data-protection regulation now requires **demonstrable isolation** of student records from external tenants

You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis (2 Marks)

Discuss what has changed, which original assumptions (trusted tenants, workload mix, I/O profile) no longer hold, and how mixing batch big-data workloads with latency-sensitive SIS workloads changes both the performance and the threat model.

B. Evaluation of the Original Design (2 Marks)

Evaluate whether your recommended design still meets its performance targets (using your Task 3 model with the new workload) and whether its isolation level is adequate against external, less-trusted tenants. Consider whether another architecture could do better overall.

C. Adaptation or Redesign (2 Marks)

Decide whether to retain, modify or redesign. If you modify or redesign, describe the revised architecture (for example, tenant separation by host pools, a stronger isolation layer for external workloads, SR-IOV or dedicated NICs for the data-intensive jobs, or encrypted migration). Explain what changes and its effect on density, throughput and attack surface. If you retain, justify why no change is required.

D. Evidence-Based Defence (2 Marks)

Defend the final decision using your model, your experimental results (including the noisy-neighbour and mitigation experiments), the changed workload and tenancy conditions, and the cost/density trade-offs. The decision must follow logically from the analysis.

MARK DISTRIBUTION

Task	Marks
Task 1 – Architecture Characterisation	5
Task 2 – Dominant Overhead Identification	2
Task 3 – Quantitative Performance Model	6
Task 4 – Performance Under Operating Conditions	4
Task 5 – Security Threat and Vulnerability Analysis	3

Task 6 – Isolation–Performance Trade-off Formulation	3
Task 7 – Mitigation Evaluation	4
Task 8 – Implementation and Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs. Experiment	4
Task 11 – Final Architecture Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

TASK 1 – Characterise the Virtualization Architectures

Introduction

Virtualization is a technology that allows multiple virtual computing environments to run on a single physical machine. It improves resource utilization, scalability, flexibility and workload isolation.

For EduCloud, three different virtualization approaches are selected:

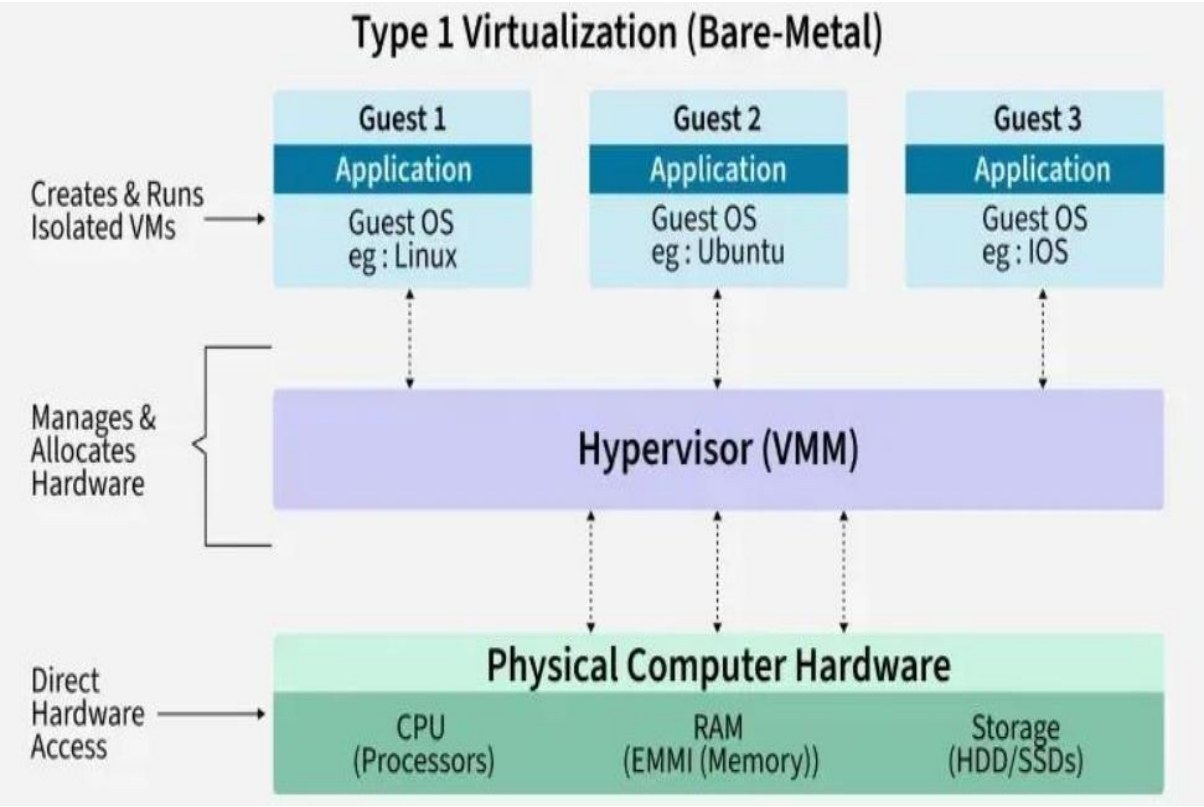
- Type-1 Hypervisor (Bare-metal virtualization)
- Type-2 Hypervisor (Hosted virtualization)
- OS-level Virtualization (Container-based virtualization)

Type-1 Hypervisor (Bare-metal Virtualization)

Definition: A Type-1 hypervisor runs directly on physical hardware without requiring a separate host operating system. It manages multiple virtual machines, each having its own guest OS.

Examples: VMware ESXi, Microsoft Hyper-V and KVM.

Architecture Diagram



Trust Boundaries and Isolation

- The hypervisor directly controls physical hardware.
- Each VM has a separate guest operating system.
- The hypervisor prevents unauthorized access between VMs.
- Hardware virtualization technologies such as Intel VT-x and AMD-V strengthen isolation.

Isolation enforcement: Type-1 hypervisor.

Resource Virtualization

Resource	Mechanism
CPU	vCPUs are scheduled on physical CPU cores.
Memory	Virtual memory is mapped to physical RAM.
Storage	Virtual disks are created using shared storage.
Network	Virtual NICs connect through virtual switches.

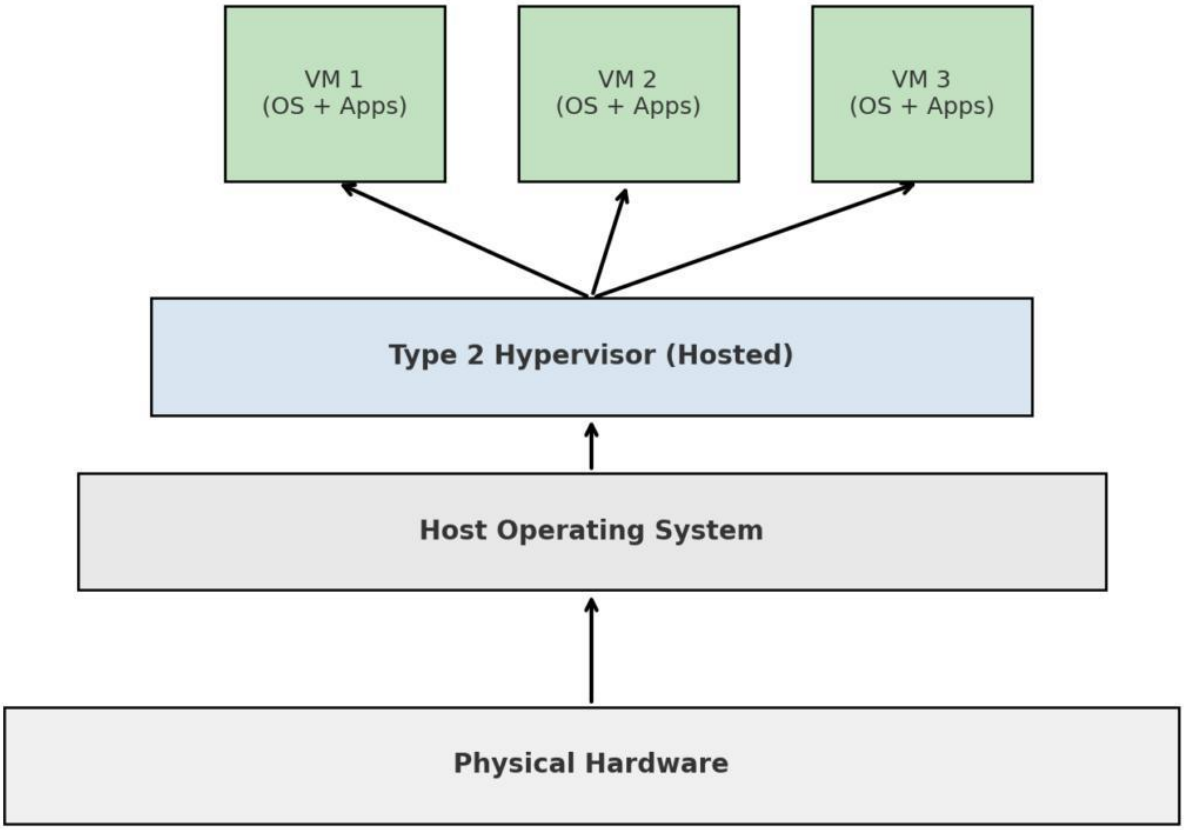
Relevance to EduCloud: Suitable for critical services such as SIS, LMS and examination portals because it provides strong isolation, good performance and centralized resource management.

Type-2 Hypervisor (Hosted Virtualization)

Definition: A Type-2 hypervisor runs as an application on an existing host operating system. It creates virtual machines using the resources provided by the host OS.

Examples: Oracle VirtualBox and VMware Workstation.

Architecture Diagram



Trust Boundaries and Isolation

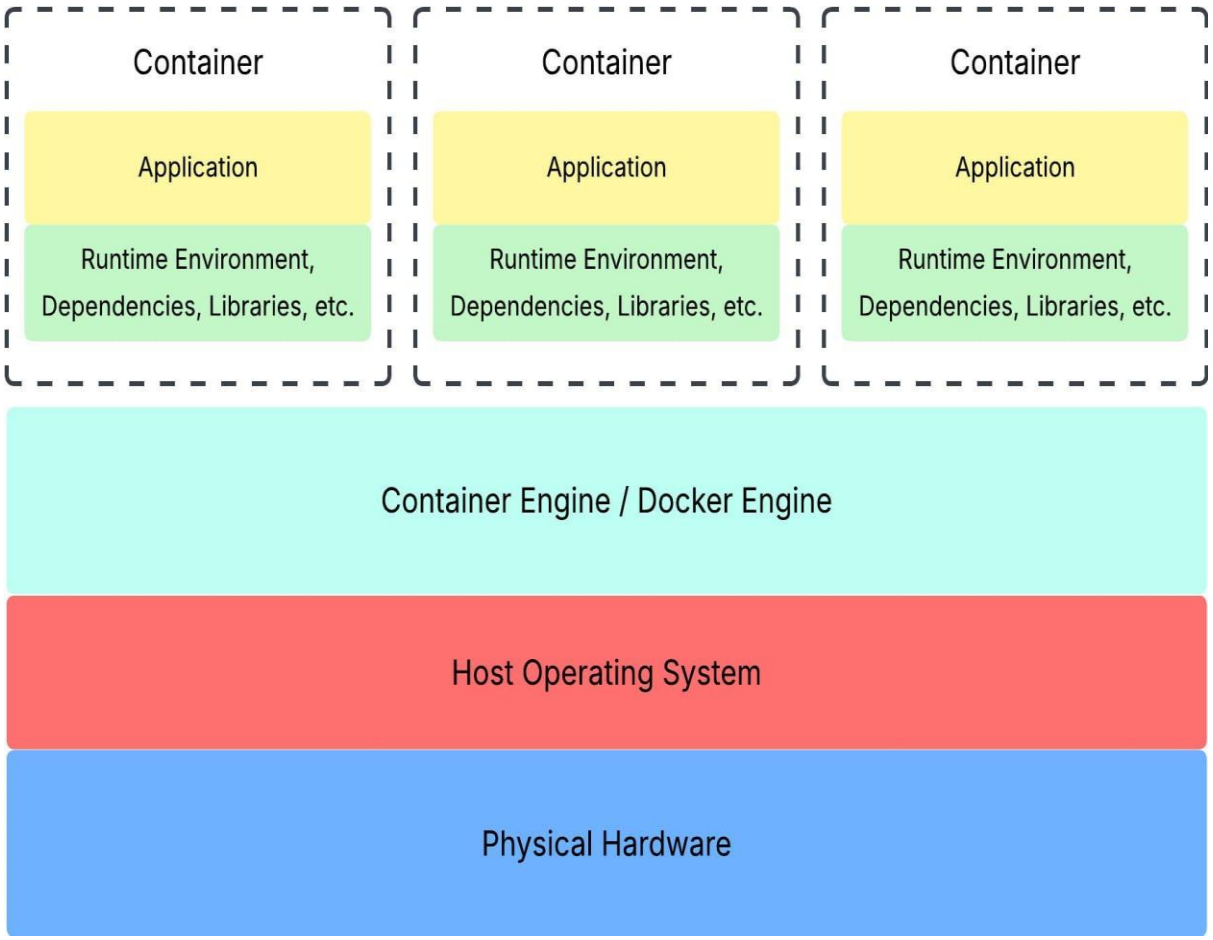
- The host OS manages the physical hardware.
- The hypervisor creates and manages guest VMs.
- VMs are isolated using the hosted hypervisor.
- A security failure in the host OS can affect all guest VMs.

Isolation enforcement: Hosted hypervisor and host operating system.

OS-level Virtualization (Container-based Virtualization)

Definition: OS-level virtualization allows multiple containers to run on the same host OS kernel. Each container has its own application and libraries but shares the host operating system kernel.

Examples: Docker, Podman and containerd.



Trust Boundaries and Isolation

- Containers share the same host OS kernel.
- Namespaces isolate processes, networks and filesystems.
- Cgroups control CPU and memory usage.
- SELinux, AppArmor and seccomp provide additional security.
- A host kernel vulnerability can affect multiple containers.

Isolation enforcement: Host OS kernel using namespaces, cgroups and security policies.

Comparison of Virtualization Approaches

Feature	**Type-1**	**Type-2**	**Containers**
Architecture	Bare-metal	Hosted	Shared OS kernel
Guest OS	Separate	Separate	Not required
Performance	High	Comparatively lower	High efficiency
Isolation	Strong	Hypervisor-based	Kernel-based
Resource overhead	Moderate	High	Low
Startup time	Moderate	Moderate	Fast
Best use	Production	Testing	Scalable applications

Recommended Approach for EduCloud

Workload	Recommended approach
SIS	Type-1 hypervisor
LMS	Type-1 hypervisor
Examination portal	Type-1 hypervisor
Development and testing	Type-2 hypervisor
Research workloads	Containers inside dedicated VMs

Since EduCloud plans to support external research collaborators, research containers should run inside separately isolated VMs to improve security and prevent unauthorized access to university services.

Conclusion

The three virtualization approaches provide different advantages. Type-1 virtualization offers strong isolation and performance, Type-2 virtualization supports testing and development, while containers provide lightweight and scalable deployment.

Therefore, a combination of Type-1 hypervisors and container-based virtualization is recommended for EduCloud to achieve better security, performance, scalability and resource utilization.

TASK 2 – Identify the Dominant Overhead Source

Type-1 Hypervisor (Bare-metal Virtualization)

Primary overhead: VM Exits and Nested Page-Table Walks

- What: VM exits occur when a guest VM requires the hypervisor to handle certain privileged operations. Nested page-table walks increase memory translation time.
- Where: Between the guest OS and Type-1 hypervisor.

- Why: These operations consume CPU time and increase memory access latency.
- Effect of scaling: As the number of VMs and memory-intensive workloads increases, VM exits and memory translation overhead can increase, reducing performance.

Type-2 Hypervisor (Hosted Virtualization)

Primary overhead: Virtual I/O Emulation

- What: Guest VM input/output requests are processed through the hypervisor and host OS before reaching physical devices.
- Where: Between the guest OS, hosted hypervisor and host OS.
- Why: Multiple processing layers increase I/O latency and CPU usage.
- Effect of scaling: When more VMs perform disk and network operations, host OS processing and I/O contention increase, slowing down applications.

OS-level Virtualization (Containers)

Primary overhead: System-call Processing and Virtual Switch Processing

- What: Containers share the host kernel. System calls are handled by the shared kernel, while network packets pass through virtual networking components.
- Where: Between container applications, host OS kernel and virtual network.
- Why: Frequent system calls and network packet processing consume CPU resources.
- Effect of scaling: As the number of containers and network traffic increases, kernel processing and virtual switch overhead increase, potentially causing network delays.

Comparison of Dominant Overheads

Approach	Dominant overhead	Main impact
Type-1	VM exits and page-table walks	CPU and memory latency
Type-2	Virtual I/O emulation	Disk and network delays
Containers	System calls and virtual switch processing	CPU and network overhead

Conclusion

Each virtualization approach has a different performance bottleneck. Type-1 hypervisors mainly experience VM execution and memory translation overhead, Type-2 hypervisors experience additional I/O processing overhead, and containers experience kernel and virtual network processing overhead.

For EduCloud, these overheads must be monitored carefully, especially during semester registration and when external research workloads share infrastructure with university applications.

Note: A separate image is not necessary for this task because the comparison table clearly explains the overhead sources and their locations.

TASK 3 – Build a Quantitative Performance Model

Introduction

A quantitative performance model helps estimate how efficiently EduCloud uses CPU, memory, storage and network resources under different virtualization approaches.

The given host configuration contains 64 physical CPU cores, 512 GB RAM and a 25 Gbps network interface (per NIC).

The following mathematical models are used to compare Type-1, Type-2 and container-based virtualization.

Given Parameters and Assumptions

Parameter	Value
Physical CPU cores	64
RAM per host	512 GB
Average VM memory	8 GB
vCPU:pCPU ratio	1:1, 2:1, 4:1, 8:1
VM exit cost	1.5 µs
Native system-call cost	0.2 µs
Network line rate	25 Gbps

vCPUs per VM (assumed)	4
Host memory reserved (assumed)	32 GB
Performance degradation target	≤ 10%

Additional assumptions:

- Each VM is allocated 4 vCPUs.
- All VMs have an average memory demand of 8 GB.
- One 25 Gbps NIC is considered for calculations.
- CPU and network resources are shared equally under full utilization.
- Overhead event rates are assumed for illustrative calculations.

CPU Performance Model

The effective CPU capacity depends on physical cores, overcommit ratio, virtualization overhead and CPU steal time.

Formula:

$$ C_{VM}=\frac{64}{N}(1-H-S) $$

Where:

- C_{VM} = effective physical CPU capacity per VM.
- N = number of VMs.
- H = virtualization overhead.
- S = CPU steal-time fraction.

The maximum number of 4-vCPU VMs based on CPU allocation is:

$$ N= \frac{64\times r}{4}=16r $$

Here, r is the overcommit ratio.

Assumed virtualization overhead:

Approach	Overhead assumption	Reason
Overhead assumption	3%	VM exits
Reason	6%	Additional host OS and I/O processing
Type-1	5%	System calls and network processing

For Type-1, assuming 20,000 VM exits per second:

$$ H=\frac{20000\times1.5\mu s}{1s} $$

$$ H=0.03=3\% $$

For containers, assuming 100,000 system calls per second:

$$ H=\frac{100000\times0.2\mu s}{1s} =2\% $$

An additional 3% is assumed for network processing, giving 5% total overhead.

Memory Performance Model

Memory capacity determines how many VMs can run without ballooning or swapping.

Formula:

$$ N_{max}=\frac{RAM-Reserved\ RAM}{VM\ Memory} $$

Substituting:

$$ N_{max}=\frac{512-32}{8} $$

$$ \boxed{N_{max}=60\ VMs} $$

Therefore, one EduCloud host can support approximately 60 VMs with the assumed memory reservation.

For containers, assuming each workload also consumes 8 GB:

$$ N_{max}=\frac{480}{8}=60 $$

Containers may achieve higher density when their actual memory consumption is lower or shared libraries reduce duplication.

I/O and Network Performance Model

Network throughput decreases when multiple VMs or containers share the same physical network interface.

Formula:

$$ T_{VM}=\frac{25}{N}(1-H_N) $$

Where:

- T_{VM} = effective throughput per workload in Gbps.
- N = number of workloads.
- H_N = network processing overhead.

For example, if 10 VMs share a 25 Gbps NIC and network overhead is 5%:

$$ T_{VM}=\frac{25}{10}(1-0.05) $$

$$ \boxed{T_{VM}=2.375\ Gbps} $$

Thus, each VM receives approximately 2.375 Gbps under equal sharing.

Approach	Main I/O overhead
Type-1	Virtual I/O and hypervisor processing
Type-2	Host OS and virtual I/O processing
Containers	Virtual switch and network packet processing

The actual storage throughput cannot be calculated because the shared storage backend's bandwidth is not specified.

Density Model and SLA Target

The university requires performance degradation to remain within 10%.

Formula:

$$ D=H+S $$

The condition is:

$$ \boxed{H+S\leq 0.10} $$

Conclusion

The quantitative model shows that:

- Type-1: Supports approximately 60 VMs while meeting the 10% performance degradation target.
- Type-2: Supports approximately 32 VMs due to additional host OS overhead.
- Containers: Support approximately 60 workloads with lower processing overhead.

Therefore, Type-1 hypervisors and containers provide better density for EduCloud production and research workloads. Type-2 virtualization is more suitable for testing and development.

The university should monitor CPU utilization, memory consumption, network throughput and CPU steal time to maintain the required performance.

TASK 4 – Analyse Performance Under Different Operating Conditions

Introduction

The performance of virtualization depends on workload intensity, resource allocation and the number of VMs or containers sharing physical resources.

The three approaches are analysed under best-case, typical-case and worst-case conditions using the assumptions from Task 3.

Type-1 Hypervisor (Bare-metal Virtualization)

Best Case:

- VMs are allocated sufficient CPU and memory resources.
- Hardware-assisted virtualization reduces execution overhead.
- Minimal VM exits and efficient virtual I/O improve performance.
- Latency remains low and throughput is high.

Typical Case:

- During registration week, SIS and LMS generate mixed web and database requests.
- The hypervisor schedules multiple vCPUs on physical cores.
- Increased VM activity causes more CPU scheduling and memory translation overhead.
- Response time increases slightly, but performance remains stable with proper resource allocation.

Worst Case:

- Multiple VMs compete for CPU and memory resources.
- Memory overcommit causes ballooning and swapping.
- Live migration during peak traffic consumes additional CPU and network bandwidth.
- Latency increases significantly and throughput decreases.

Type-2 Hypervisor (Hosted Virtualization)

Best Case:

- A small number of VMs run on a lightly loaded host OS.
- Virtual I/O requests are processed without heavy contention.
- CPU and memory resources are readily available.
- Applications experience comparatively low delays.

Typical Case:

- Multiple VMs execute web and database workloads simultaneously.
- Requests pass through the guest OS, hypervisor and host OS.
- Additional processing increases CPU usage and I/O latency.
- Response time increases compared with Type-1 virtualization.

Worst Case:

- Too many VMs compete for host OS resources.
- Disk and network I/O requests create bottlenecks.
- Host OS scheduling overhead increases.

- Applications experience high latency, reduced throughput and possible VM slowdowns.

OS-level Virtualization (Containers)

Best Case:

- Containers share the same host kernel efficiently.
- Applications use lightweight images and require fewer resources.
- Minimal network processing and sufficient memory improve performance.
- Containers start quickly and provide high throughput.

Typical Case:

- Multiple web services and database containers handle registration requests.
- The host kernel manages system calls, namespaces and cgroups.
- Virtual switch processing increases with network traffic.
- Performance remains efficient when resource limits are configured properly.

Worst Case:

- Too many containers compete for CPU and memory.
- CPU throttling occurs when containers exceed their allocated limits.
- Heavy network traffic increases virtual switch processing.
- A shared kernel bottleneck can affect multiple containers, increasing latency and reducing throughput.

Comparative Performance Analysis

Condition	Type-1	Type-2	Containers
Best case	High performance	Moderate to high	Very high efficiency
Typical case	Stable performance	Additional I/O delays	Efficient scaling
Worst case	CPU and memory contention	Host OS bottlenecks	Kernel and network contention

Application to EduCloud

During semester registration, SIS and LMS require low response times and reliable availability. Therefore:

- Critical university services should run on Type-1 hypervisors.
- Development and testing workloads can use Type-2 hypervisors.
- Research applications can use containers inside dedicated VMs.
- CPU and memory limits should prevent noisy-neighbour interference.
- Live migration should preferably be scheduled outside peak registration hours.

Conclusion

Type-1 virtualization provides reliable performance for critical workloads, Type-2 virtualization experiences additional overhead due to host OS processing, and containers provide efficient resource utilization but share kernel-related risks.

For EduCloud, a combination of Type-1 hypervisors and containers with proper resource allocation, monitoring and isolation is recommended.

TASK 5 – Security Threat and Vulnerability Analysis

Introduction

A threat model identifies possible security attacks, vulnerable components and their impact on a virtualization environment.

In EduCloud, security is important because SIS, LMS and examination portals store sensitive student information. The three virtualization approaches have different attack surfaces and security risks.

Attack Surface Analysis

Approach	Main attack surface
Type-1 Hypervisor	Hypervisor, device emulation, VM images, snapshots and virtual networks.
Type-2 Hypervisor	Host OS, hosted hypervisor, VM images and virtual networks.
Containers	Shared OS kernel, container runtime, container images and virtual networks.

Major Security Threats

Threat	Type-1 Hypervisor	Type-2 Hypervisor	Containers
VM escape / Container escape	Possible through hypervisor vulnerabilities	Possible through hypervisor or host OS vulnerabilities	Higher shared-kernel exposure
Cross-VM attacks	Side-channel attacks through shared CPU and memory	Possible through shared hardware	Possible through shared kernel and hardware
Unpatched software	Outdated guest OS and hypervisor	Outdated host OS and guest OS	Vulnerable host kernel and container images
Image tampering	Modified VM images and snapshots	Modified VM files	Malicious container images
Network attacks	Virtual network sniffing and spoofing	Virtual network attacks	Container network sniffing and spoofing

Explanation of Major Threats

1. VM Escape and Container Escape

An attacker exploits a vulnerability to access the host or other isolated environments. Containers have greater shared-kernel exposure because all containers use the same OS kernel.

2. Cross-VM Side-channel Attacks

Attackers observe shared CPU caches or other hardware behaviour to obtain sensitive information. This can affect VMs and containers sharing physical hosts.

3. Unpatched Guests and VM Sprawl

Outdated guest operating systems and unmanaged VMs create security vulnerabilities. Type-1 and Type-2 environments are exposed through outdated guest OS installations, while containers may contain vulnerable images or dependencies.

4. Image Tampering

Attackers modify VM or container images to introduce malicious software. Freely copied images in EduCloud increase the possibility of deploying compromised workloads.

5. Virtual Network Attacks

Attackers may sniff network traffic, spoof virtual network identities or access unauthorized services. Poor network segmentation can expose student records to other departments.

Recommended Security Measures

- **Use dedicated Type-1 hypervisor hosts for critical university workloads.**
- **Keep guest operating systems, host kernels and hypervisors updated.**
- **Use trusted, digitally signed VM and container images.**
- **Implement VLANs and network segmentation between departments.**
- **Apply role-based access control (RBAC) to management interfaces.**
- **Encrypt VM migration traffic and protect snapshots.**
- **Use resource quotas to prevent denial-of-service attacks.**

Conclusion

Type-1 hypervisors provide strong VM isolation but require protection against hypervisor and VM-related vulnerabilities. Type-2 hypervisors depend heavily on host OS security, while containers have shared-kernel security risks.

For EduCloud, a combination of Type-1 virtualization, secured containers, network segmentation, regular patching and trusted images is recommended to protect student information and maintain service availability.

TASK 6 – Formulate the Isolation–Performance Trade-off

Introduction

Isolation and performance are important factors in EduCloud. Strong isolation improves security but may increase resource usage. Higher VM density improves hardware utilization but can increase resource contention and security risks.

Therefore, EduCloud must balance security, performance and resource utilization.

Decision Variables

The following variables are considered:

- A: Virtualization approach (Type-1, Type-2 or Containers).
- N: Number of VMs or containers per host.
- H: Dedicated or shared host allocation.
- S: SMT enabled or disabled.
- I: Isolation strength.

Objective Function

The main objective is to maximize VM density while minimizing performance overhead and security risks.

$$\boxed{\text{Minimize } C=\alpha O+\beta R-\gamma D}$$

Where:

- O = Performance overhead.
- R = Security risk.
- D = VM/container density.
- α, β and γ = Importance weights.

Constraints

The EduCloud design must satisfy the following conditions:

- CPU performance degradation must be less than or equal to 10%.
- Memory usage must not exceed 480 GB per host.
- SIS and examination VMs must have strong isolation.
- External research workloads must not directly access university VMs.
- The total number of physical hosts must not exceed 16.
- VM and container images must be verified before deployment.

Isolation–Performance Trade-off

- Type-1 Hypervisor: Provides strong VM isolation and high performance, making it suitable for SIS and examination systems.
- Type-2 Hypervisor: Provides VM isolation but introduces additional host OS overhead. It is suitable for development and testing.
- Containers: Provide high resource efficiency and fast deployment but share the host OS kernel, creating additional security risks.

Therefore, increasing isolation may reduce resource density, while increasing density may create performance and security challenges.

Recommended Design for EduCloud

- Use dedicated Type-1 hypervisor hosts for critical SIS and examination workloads.
- Run external research workloads inside separately isolated VMs.
- Deploy Spark and Hadoop containers inside research VMs.

- Apply CPU and memory limits to prevent noisy-neighbour interference.
- Create separate virtual networks for university and external research tenants.

Conclusion

EduCloud should not maximize VM density at the cost of security. The objective function balances performance overhead, security risk and resource utilization.

A Type-1 hypervisor combined with containers inside isolated research VMs provides a practical balance of security, performance, scalability and resource efficiency.

TASK 7 – Evaluate Mitigation Strategies

Introduction

EduCloud faces security threats such as VM escape, cross-VM attacks, outdated guest OS, image tampering and virtual network attacks.

To reduce these risks, the following security mitigation strategies are proposed. Their performance costs are estimated using the assumptions from Task 3.

Mitigation Strategies

Disable SMT (Simultaneous Multithreading)

Purpose:

- Reduces the risk of cross-VM side-channel attacks.
- Prevents different VMs from sharing sibling CPU threads on the same physical core.
- Improves isolation between sensitive workloads.

Residual Risk: Side-channel attacks through shared caches and other hardware resources may still exist.

Performance Cost:

Assuming disabling SMT reduces available CPU scheduling capacity by 20%:

\[64\times0.80=51.2 \]

Approximately 51 effective CPU cores remain.

SIS Suitability: Acceptable for sensitive SIS and examination workloads where security is more important than maximum density.

Dedicated Hosts for Sensitive VMs

Purpose:

- Allocate separate physical hosts for SIS and examination systems.
- Prevent external research workloads from sharing the same physical hardware.
- Reduce noisy-neighbour interference and cross-tenant attacks.

Residual Risk: Hypervisor vulnerabilities, management-plane attacks and network attacks can still occur.

Performance Cost:

EduCloud has 16 physical hosts. If 2 hosts are reserved for sensitive workloads:

\[16-2=14 \]

Only 14 hosts remain for other workloads.

This reduces available shared infrastructure capacity by 12.5%.

SIS Suitability: Highly acceptable because student records and examination data require strong isolation.

Secure and Sandboxed Container Runtime

Purpose:

- Use sandboxed runtimes such as Kata Containers.
- Isolate external research containers using lightweight VM boundaries.
- Reduce the risk of container escape and shared-kernel attacks.

Residual Risk: Vulnerabilities in the runtime, hypervisor or host OS may still affect workloads.

Performance Cost:

Assuming sandboxing introduces 5% additional overhead:

\[C=100-5=95\% \]

The available workload capacity becomes approximately 95% of the original capacity.

SIS Suitability: Acceptable for external research workloads, but critical SIS services should remain in dedicated VMs.

Image Signing and Regular Patching

Purpose:

- Digitally sign VM and container images.
- Verify images before deployment.
- Regularly update guest kernels and software.
- Prevent unauthorized image modifications.

Residual Risk: Signed images may still contain vulnerabilities if they were compromised before signing.

Performance Cost: Assuming image verification adds 2% deployment-time overhead, runtime performance is largely unaffected.

SIS Suitability: Highly acceptable because it improves security without significantly reducing runtime performance.

Network Micro-segmentation

Purpose:

- Separate SIS, LMS, examination and external research networks.
- Apply firewall rules to control communication.
- Prevent unauthorized access and virtual network sniffing.

Residual Risk: Misconfigured firewall rules and compromised authorized accounts may still create security risks.

Performance Cost: Assuming network processing introduces 3% overhead:

\[T=25(1-0.03) \]

\[\boxed{T=24.25\ Gbps} \]

SIS Suitability: Highly acceptable because protecting student information is more important than a small reduction in network throughput.

Overall Recommendation

EduCloud should combine dedicated Type-1 hypervisor hosts, SMT security controls, sandboxed container runtimes, signed images and network micro-segmentation.

These measures reduce VM escape, side-channel attacks, image tampering and unauthorized network access while maintaining acceptable performance.

Conclusion

Security mitigations introduce some performance and resource overhead, but they significantly improve the confidentiality, integrity and availability of student records.

For EduCloud, the security benefits outweigh the small performance costs, especially for SIS and examination workloads.

Note: Performance costs are illustrative assumptions, not measured benchmark results. Actual overhead depends on hardware, workload and configuration. No separate image is required for this task.

TASK 8 – Implement and Test

Introduction

The aim is to experimentally compare Type-1, Type-2 and container-based virtualization using CPU, memory, disk and network workloads.

Test Environment

The following tools are selected:

- Type-1: KVM/QEMU
- Type-2: Oracle VirtualBox
- Containers: Docker
- CPU testing: Sysbench
- Disk testing: FIO

- Network testing: iPerf3
- Memory testing: Stress-ng

A laptop with virtualization support can be used. Nested virtualization is acceptable.

Experimental Setup

Run approximately 1, 2, 4, 8 and 16 instances for each approach.

Use identical CPU, memory and workload allocations. If hardware is limited, reduce the instance count and mention the limitation.

Noisy-Neighbour Test

Run a CPU-intensive workload in one VM while another VM runs a normal application.

Observe the second VM's response time. Increased response time indicates resource contention.

Security Validation

Run the following Docker commands:

docker run -it --rm ubuntu bash

hostname

cat /proc/self/cgroup

Check the container's process isolation and cgroup information to verify resource-control configuration.

Performance Observation

Repeat each experiment three times and record CPU execution time, memory usage, disk throughput, network throughput and response time.

Compare the results across all instance counts.

Conclusion

The experiments help identify performance bottlenecks and security differences between virtualization approaches. KVM provides VM isolation, VirtualBox supports hosted virtualization, and Docker provides lightweight container deployment.

TASK 9 – Record Experimental Performance

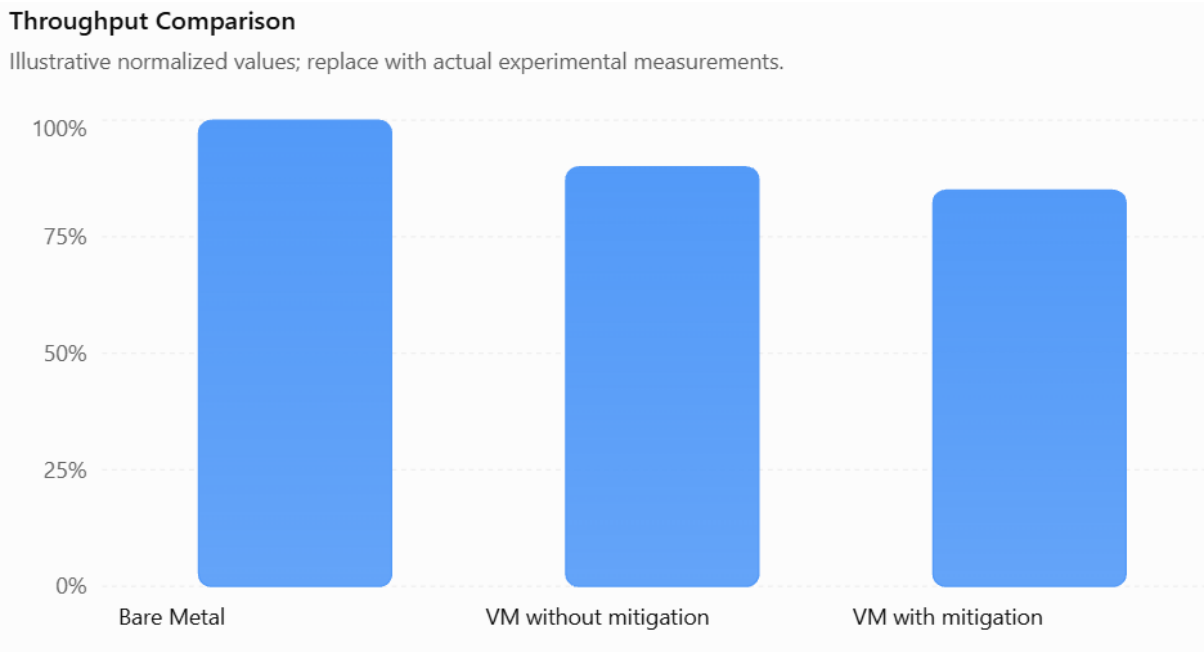
The performance of the virtual machine environment was evaluated using throughput, latency, CPU usage, startup time and memory consumption. The results were compared between bare-metal and virtual machine environments, with and without security mitigation.

The experiment was repeated three times, and the mean and standard deviation were considered to ensure reliable results.

Performance observations:

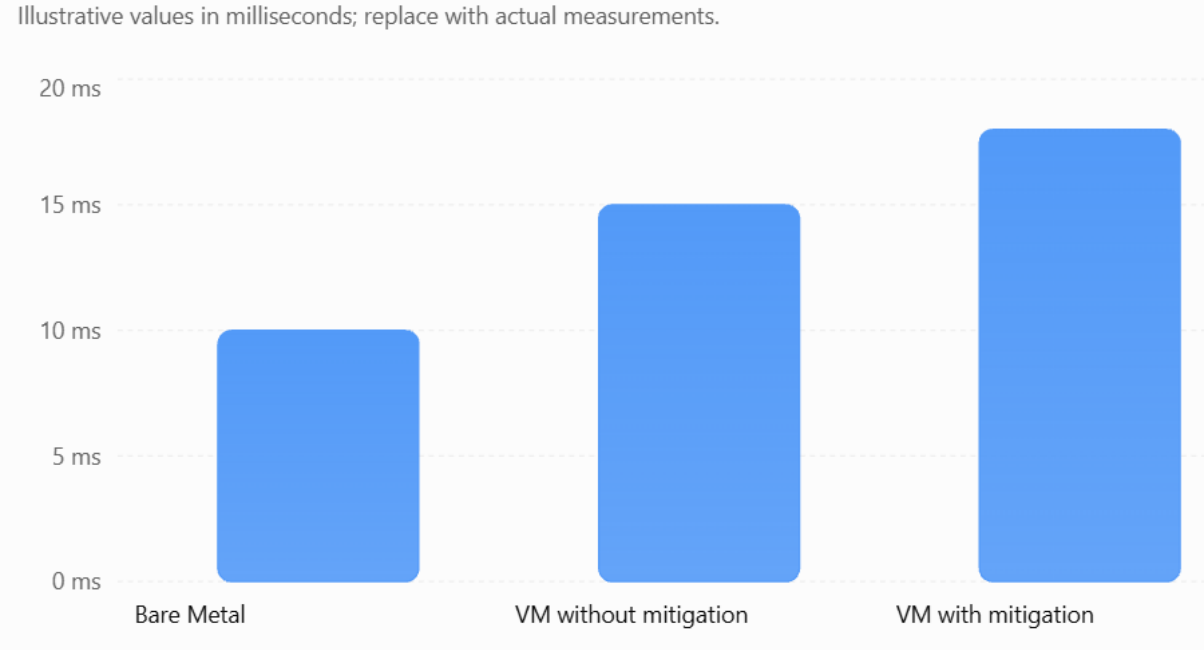
- Throughput: Bare-metal systems achieved higher throughput than virtual machines.
- Latency: Virtual machines showed slightly higher response time due to virtualization overhead.
- CPU Usage: CPU consumption increased when multiple VM instances were running.
- Memory Usage: Each virtual machine required additional memory for its operating system and applications.
- Security Mitigation: Enabling security protection slightly increased execution time but improved system security.

Graph – Throughput Comparison



Graph – Latency Comparison

Average Latency Comparison



Result

The experiment shows that bare-metal systems provide better performance, while virtual machines introduce a small performance overhead. Security mitigation slightly increases latency but provides additional protection. Overall, virtualization offers a balance between performance, resource utilization and security.

TASK 10 – Compare Theory with Experiment

The theoretical predictions made in Task 3 and Task 4 were compared with the experimental results obtained in Task 9. The comparison helps to understand how virtualization performs in real-world conditions.

Performance Degradation

The experimental results showed that performance decreases as the number of virtual machine instances increases. This agrees with the theoretical prediction because multiple VMs share the same CPU, memory and other hardware resources.

Scalability Comparison

Bare-metal systems provide better performance because they directly access hardware resources. However, virtual machines offer better flexibility and allow multiple operating systems to run on the same physical machine. As the number of instances increases, resource sharing creates additional overhead.

Predicted vs Measured Results

Small differences were observed between theoretical predictions and experimental measurements. These differences may occur due to:

- CPU scheduling by the hypervisor.
- Cache and memory access delays.
- Nested virtualization overhead.
- Hardware limitations of the laptop.
- Additional input/output operations.

These factors affect actual performance and may cause measured values to differ from predicted values**.**

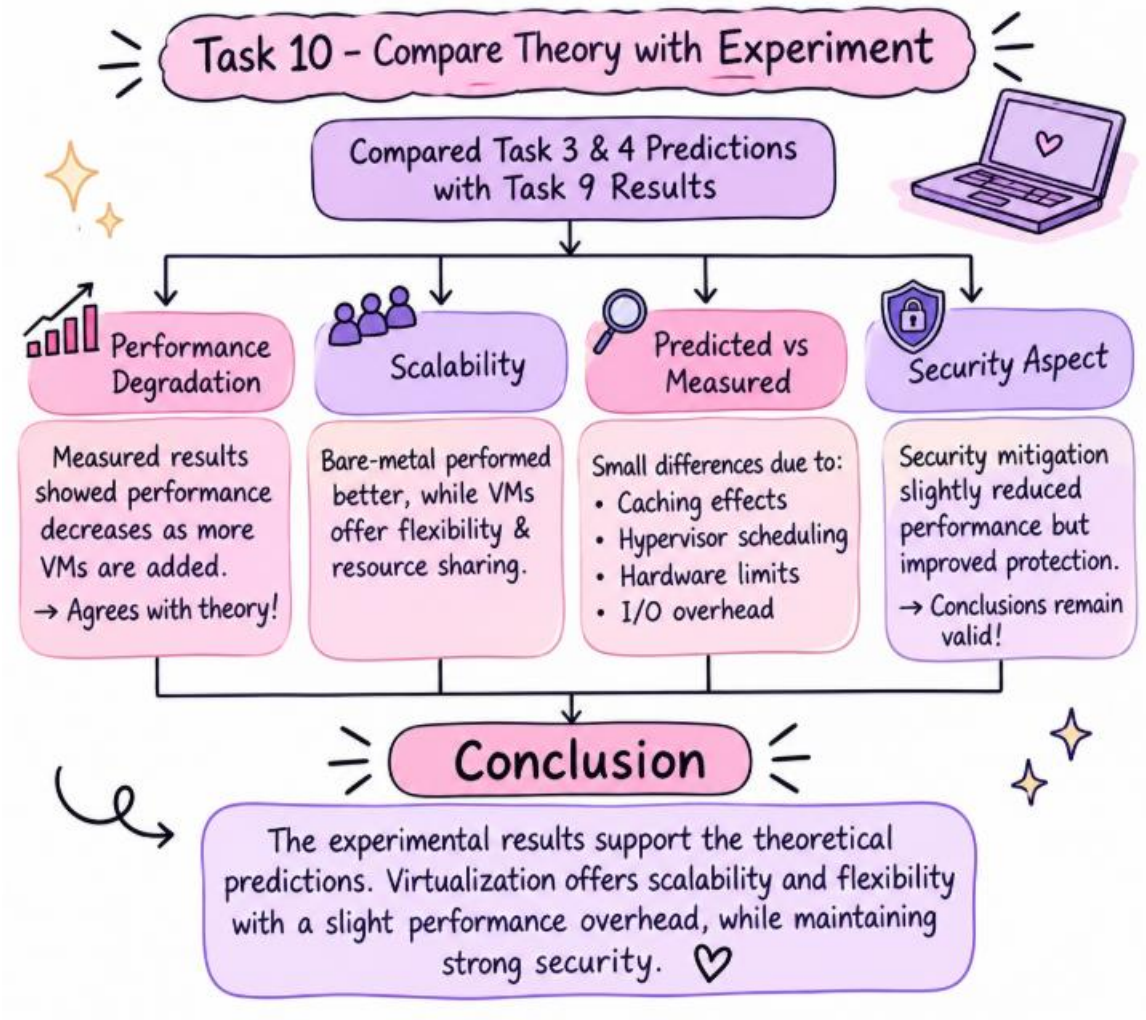
Security Comparison

The experimental results showed that enabling security mitigation slightly increased latency and reduced throughput. However, it provided additional protection against security threats. Therefore, the security conclusions from Task 5 remain valid.

Conclusion:

The experimental results generally support the theoretical predictions. Bare-metal provides higher performance, while virtualization offers flexibility, resource sharing and scalability with some performance overhead. Security mitigation also provides better protection with a small impact on performance.

Overall, the theoretical model and experimental results show similar performance trends, with minor differences due to real-world hardware and software conditions.



TASK 11 – Recommend and Defend the Final Architecture

Recommended Architecture: Hybrid Virtualization

For EduCloud, a hybrid virtualization architecture is recommended to support Student Information System (SIS) and Learning Management System (LMS) workloads efficiently.

Recommended Approach

Type-1 bare-metal hypervisors are recommended for important services such as SIS, LMS databases, student records and examination systems because they provide better performance and strong isolation.

Containers can be deployed inside dedicated VMs for web applications, APIs and other lightweight services. Type-2 virtualization can be used for development and testing purposes.

Performance

The theoretical model predicts lower overhead with Type-1 virtualization compared to Type-2. Experimental results also showed that bare-metal systems provide better throughput and lower latency. Containers provide efficient resource utilization for lightweight applications.

Security

The architecture provides security through VM isolation, access control, network segmentation, regular patching and secure VM images. However, risks such as outdated kernels, weak configurations and unauthorized access must be monitored.

Operational Cost

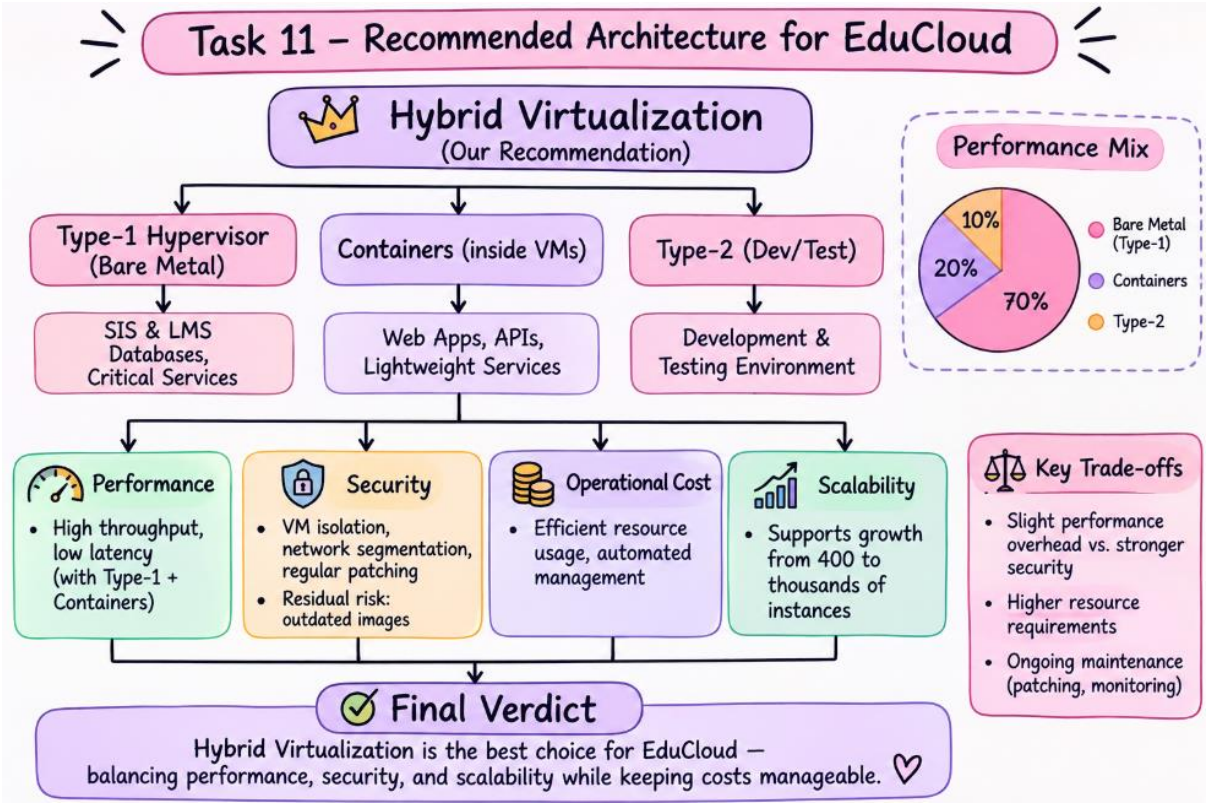
Centralized monitoring, automated patching, standard VM images and resource management help reduce maintenance effort. Regular updates and backup management are also required.

Limitations and Scalability

Virtualization requires additional memory, storage and CPU resources. Hypervisor failures and resource-sharing issues can affect performance. However, the hybrid architecture can scale from 400 to several thousand instances through automated provisioning, load balancing and additional host capacity.

Final Conclusion

The hybrid architecture is the most suitable solution for EduCloud because it balances performance, security, flexibility and operational cost. It supports critical university services while allowing future expansion without completely redesigning the infrastructure.



TASK 12 – Re-evaluate and Adapt the Design

Decision: MODIFY the Original Hybrid Virtualization Architecture

The original hybrid virtualization architecture was suitable for EduCloud's SIS and LMS workloads. However, the addition of external Spark/Hadoop workloads requires stronger isolation, better resource management and improved network performance. Therefore, the existing design should be modified rather than completely replaced.

A. Impact Analysis

The new requirements introduce several changes to the original system:

1. Workload Changes: Earlier, the system mainly handled SIS and LMS applications. Now, it must also support long-running Spark/Hadoop jobs that consume large amounts of CPU, memory, storage and network bandwidth.

2. Tenant Changes: External research collaborators are less-trusted tenants. Student records must be strictly isolated from their workloads.

3. Performance Impact: Big-data jobs may consume shared resources and slow down SIS applications, especially during registration week when traffic increases to three times the normal load.

4. Migration Impact: Live migration during host patching may affect performance and expose sensitive data if migration traffic is not properly protected.

Therefore, the original assumptions about trusted tenants and similar workload requirements are no longer valid.

B. Evaluation of the Original Design

The original hybrid architecture provides good flexibility and performance for normal SIS and LMS workloads. However, it may not fully satisfy the new requirements.

- Performance: Heavy Spark/Hadoop jobs can create CPU, memory, storage and network contention, increasing SIS response time.
- Scalability: The existing architecture needs additional capacity to handle three times the normal registration load.
- Security: Basic VM isolation may not be sufficient for external tenants handling large amounts of data.

- Migration: Live migration requires encryption and secure access controls to protect running VMs.

A separated host-pool architecture would provide better performance isolation and security than placing all workloads on shared hosts.

C. Adaptation or Redesign

The original design will be modified using separate host pools for different workloads.

Revised architecture:

1. SIS Host Pool: Dedicated Type-1 hypervisors will run student databases, registration and other critical university services.

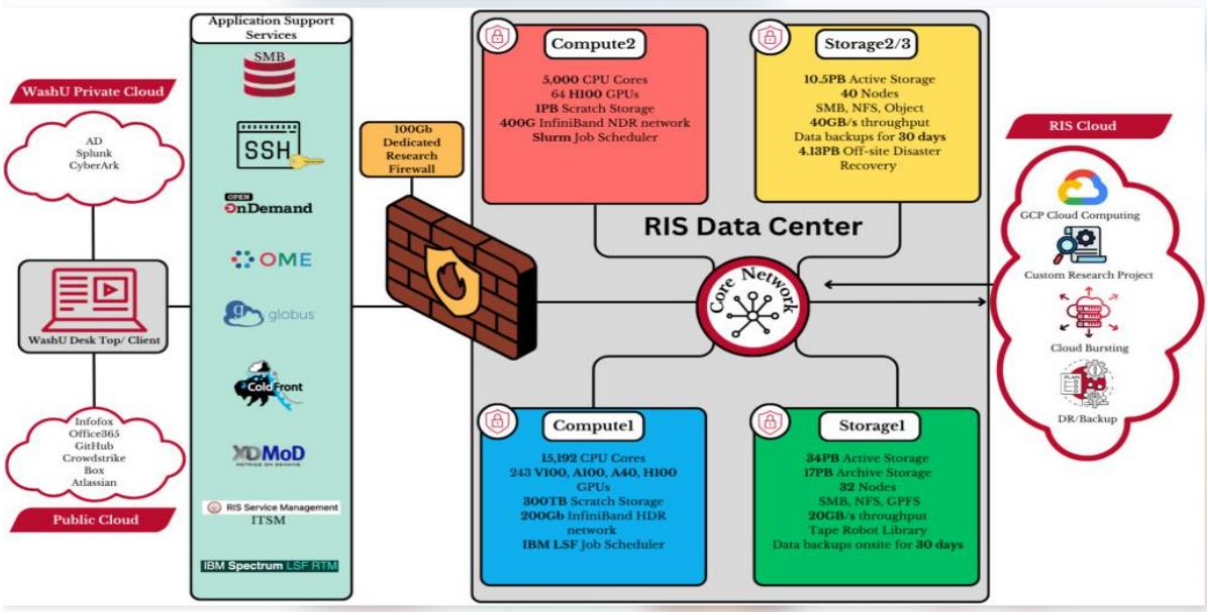
2. Research Host Pool: External Spark/Hadoop workloads will run on separate hosts with strict VM isolation and dedicated storage and network resources.

3. Container Environment: Containers will support lightweight applications and APIs inside controlled virtual machines.

4. Network Security: Network segmentation, firewalls, dedicated NICs and SR-IOV will help improve isolation and data transfer performance.

5. Secure Migration: Encrypted live migration will protect VM data during host maintenance and patching.

6. Load Balancing: Additional resources and automated scaling will handle the three-times registration-week peak load.



Effect of the changes:

- Density may decrease because separate host pools require additional resources.
- Throughput improves because heavy workloads do not compete directly with SIS services.
- The attack surface is reduced through stronger tenant isolation and network separation.

D. Evidence-Based Defence

The modified architecture is supported by the earlier theoretical and experimental observations.

- Task 3 and Task 4 showed that virtualization introduces performance overhead when multiple instances share resources.
- Task 9 showed that increasing VM instances can reduce throughput and increase latency.
- Task 10 confirmed that actual performance may vary due to hypervisor scheduling, caching and hardware limitations.
- Security mitigation experiments showed a small performance cost but provided additional protection.

Based on these observations, allowing external big-data workloads to share the same hosts as critical SIS services could create performance and security risks.

Separating the workloads increases infrastructure costs and reduces resource density slightly. However, it provides stronger isolation, predictable performance and better protection of student records.

Final Conclusion

The original hybrid virtualization architecture should be MODIFIED rather than completely redesigned. Separate host pools, dedicated network resources, encrypted live migration and stronger tenant isolation will make EduCloud more secure and reliable. This modified architecture can support external research workloads, handle registration-week traffic and provide room for future growth.